1.INTRODUCTION

The Placement Cell App is a comprehensive mobile application developed in Flutter, designed to streamline the placement process for students, volunteers, and placement coordinators. The app aims to address the challenges faced in managing the placement activities efficiently. It provides an intuitive and user-friendly interface for students to create detailed profiles, track their application status, and receive updates on job opportunities. Volunteers play a vital role in assisting students by sharing job openings, verifying their details, and offering guidance for job interviews. The app also facilitates effective communication between volunteers and students. The placement coordinator is responsible for managing job postings from various companies, including salary packages, job profiles, and other relevant information. The coordinators can view and manage student details, including resumes and application status. The project follows an incremental development process model, allowing for iterative improvements and feedback at each stage. The chosen technology stack includes Flutter for frontend development, Node.js for backend development, and MongoDB as the database. The use of Flutter ensures a cross-platform application with a native-like user experience, while Node.js provides a scalable and efficient backend. MongoDB is a robust and flexible database solution for storing and managing data. The Placement Cell App aims to improve the efficiency and effectiveness of the placement process, benefiting students, volunteers, and placement coordinators alike. By providing a centralized platform for communication, information sharing, and data management, the app simplifies the complexities involved in the placement process and enhances the overall experience for all stakeholders.

2. SYSTEM ANALYSIS

System analysis is the process of gathering and interpreting facts, diagnosing problems and using the facts to improve the system. Analysis is a detailed study of various operations performed by a system and their relationship within and outside of the system . This involves gathering information and using structured tools for analysis. System analysis is a step-by-step process used to identify and develop or acquire the software need to control the processing of specific application. System analysis is a continuing activity the stages of the systems development. System analysis is the process of gathering and interpreting facts, diagnosing problems and using the facts to improve the system. The outputs from the organization are traced through the various processing that the input phases through in the organization. This involves gathering information and using structured tools for analysis. A detailed study of this process must be made by various techniques like interviews; questionnaires etc. It is necessary to have such a good system analysis and then by a project development cycle so that the project can be completed in a strictly manner and able to finish with the desired time. The analyst must be so careful about his responsibilities. As the next step the current system analysis is done which identifies the real need of establishing our project in the environment, its opportunities and constraints etc. All of the steps discussed above are collectively known as the system analysis.

2.1. EXISTING SYSTEM

The traditional placement process involves a lot of paperwork, manual communication, and delays in the exchange of information between students, volunteers, and coordinators. This system is often inefficient and can lead to miscommunication and confusion. Students may struggle to keep track of their application status and may miss out on job opportunities. Volunteers may find it challenging to verify student details and offer timely assistance in preparing for job interviews. Coordinators may face difficulty managing job postings and communicating with students effectively.

Limitation Of Existing System

- Limited Features
- User Experience Issues
- Data Privacy Concerns
- Lack of Customization
- Reliability and Maintenance
- Limited Availability

2.2PROPOSED SYSTEM

To address the limitations of the existing system, the Placement Cell App has been developed using Flutter, which is a cross-platform development tool. This app provides a comprehensive solution for the placement process, making it easier and more efficient for all stakeholders involved. Students can create their profiles, track their application status, and receive updates on upcoming job opportunities. Volunteers can verify student details, offer assistance in preparing for job interviews, and inform students about job openings. Coordinators can manage job postings, communicate with students effectively, and view and manage student details, including their resumes and application status. Overall, the Placement Cell App offers a user friendly and streamlined solution for the placement process, enhancing communication and ensuring a successful placement experience for all.

Advantage of Proposed system

- Comprehensive Features
- Enhanced User Experience
- Improved Access to Services
- Personalization Options
- Data Privacy and Security
- Reliability and Maintenance

2.3 SYSTEM REQUIREMENT SPECIFICATION

A software requirements specification (SRS) is a comprehensive description of the intended purpose and environment for software under development. The SRS fully describes what the software will do and how it will be expected to perform. An SRS minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost. A good SRS defines how an application will interact with system hardware, other programs and human users in a wide variety of real-worked situations.

2.3.1 Hardware Specifications

Processor : INTEL Core i3

Speed : 3.0GHz or higher

System bus : 64bits

Memory : 8GB RAM

Hard disk : 512GB

Monitor : 15.6" LCD Monitor

Keyboard : 108 keys

2.3.2 Software Specifications

Operating System : Android, Ios

Front End : Flutter

Back End : Node.js

Designing tools : Figma

Browser : Chrome

Database : MongoDB

2.3.3 Front end

The front-end of the Placement cell App will be developed using Flutter, which is a popular and powerful open-source UI toolkit. Flutter is developed by Google and allows for the creation of high-quality native interfaces for both Android and iOS platforms from a single codebase. It is well-known for its fast development speed, expressive and flexible UI components, and hot reload feature, which enables developers to see changes instantly without restarting the app.

Flutter uses the Dart programming language, which is designed for building mobile, web, and server applications. The rich set of pre-designed widgets and customizability provided by Flutter make it an excellent choice for developing visually appealing and responsive user interfaces. Additionally, Flutter has a large and active community, which means plenty of resources, plugins, and libraries are available to aid in app development.

The decision to use Flutter as the front-end tool for the Placement cell App ensures that the app will have a consistent and native-like user experience on both Android and iOS devices. Its ease of use, performance, and cross-platform capabilities make it a popular choice for developing modern mobile applications.

2.3.4 Back end

The back-end of the Placement cell App will be developed using Node.js, a popular and versatile JavaScript runtime environment. Node.js allows for building scalable, high-performance server-side applications using JavaScript, which enables developers to work on both the front-end and back-end of the app using the same programming language.

Node.js excels in handling concurrent connections, making it well-suited for apps that require real time interactions, such as attendance tracking and data synchronization. It uses an event-driven, non-blocking I/O model, which enhances the app's efficiency and responsiveness. Additionally, Node.js has a vast ecosystem of modules and libraries available through n p m (Node Package Manager), allowing developers to integrate various functionalities seamlessly.

The decision to use Node.js for the back-end of the Placement cell App ensures that the app can handle a large number of users and provide real-time updates without compromising on

performance. Its ability to handle asynchronous tasks and event-driven nature makes it suitable for building responsive and scalable server applications.

Node.js will be responsible for managing data processing, user authentication, and interactions with the MongoDB database, where attendance records and other relevant data will be stored. The back-end will also handle the logic for generating attendance analytics, sending notifications, and implementing role-based access control to ensure secure and efficient data management. Overall, the use of Node.js as the back-end technology for the Placement cell App complements the front-end Flutter framework, creating a powerful and efficient full-stack solution that can meet the needs of Placement cell App.

2.4 FEASIBILITY ANALYSIS

A feasibility study is an evaluation and analysis of the potential of the proposed project which is based on extensive investigation and research to give full comfort to the decision makers. Feasibility studies aim to objectively and rationally uncover the strength and weakness of existing business of proposed venture, opportunities and threads as presented by the environment, the resources required to carry through, and ultimately the process for success. In its simplest terms, the two criteria to judge feasibility are cost required and value to attain. As such, a well-designed feasibility study should provide a historical background of the business or project, description of the product or service, accounting statements, details of the operations and management, marketing research and policies, financial data, legal requirements and tax obligations.

The two aspects in the feasibility study are:

- Technical feasibility
- Operational feasibility

Technical Feasibility

The technical feasibility centres on the existing system and what extend it can support the proposed addition. The technical feasibility assessment is focused on gaining an understanding of the present technical resources of the organization and their applicability to the expected needs of the proposed system. The minimum requirements of the system are met by average user. The developer system has á modest technical requirement as only minimal or null changes are required for implementing system.

Normally associated with the technical feasibility includes:

• Development risk

• Resource availability

• Technology

The proposed system can work without any additional hardware or software support other than the computer system and networks. So, I analysed that the proposed system is much more technically feasible than other systems when comparing with the benefits of the new system.

Operational Feasibility

Operational feasibility is a measure of how well a proposed system solves the problems, and takes advantage of the opportunities identified during scope definition and how it satisfies the requirements identified in the requirements analysis phase of system development.

2.5 DATA FLOW DIAGRAM (DFD)

Introduction to data flow diagram A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system. It differs from the flowchart as it shows the data flow instead of the control flow of the program. A data flow diagram can also be used for the visualization of data processing (structured design). Data flow diagrams were invented by Larry Constantine, the original developer of structured design. based on Martin and Estrin's "data flow graph" model of computation.

Data flow diagrams (DFDs) are one of the three essential perspectives of Structured System Analysis and Design Method SSADM. The sponsor of a project and the end users will need to be briefed and consulted throughout all stages of a system's evolution. With a data flow diagram, users are able to visualize how the system will operate, what the system will accomplish. and how the system will be implemented. The old system's data flow diagrams can

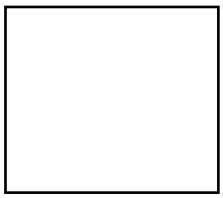
be drawn up and compared with the new system's data flow diagrams to drawn comparisons to implement a more efficient system. Data flow diagrams can be used to provide the end user with physical idea of where the data they input ultimately has an effect upon the structure of the whole system from order to dispatch to report. How any system is developed can be determined through a data flow diagram.

Developing a data flow diagram helps in identifying the transaction data in the data model. There are different notations to draw data flow diagrams, defining different visual representation for process, data stores, data flow, and external entities. The first step is to draw a data flow diagram (DFD). A DFD also known as "bubble chart" has the purpose of clarifying system requirements and identifying major transformation that will become program in system design. So, it is starting point of the design phase that functionally decompose the requirements specification down to the lowest level of details DFD consists of

series of bubbles joined by lines. The bubbles represent data transformation and the Iines represent data flow in the system.

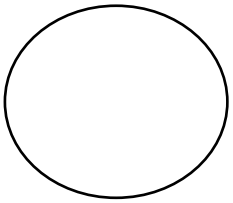
DFD Symbols

Square- Defines source or destination of system.



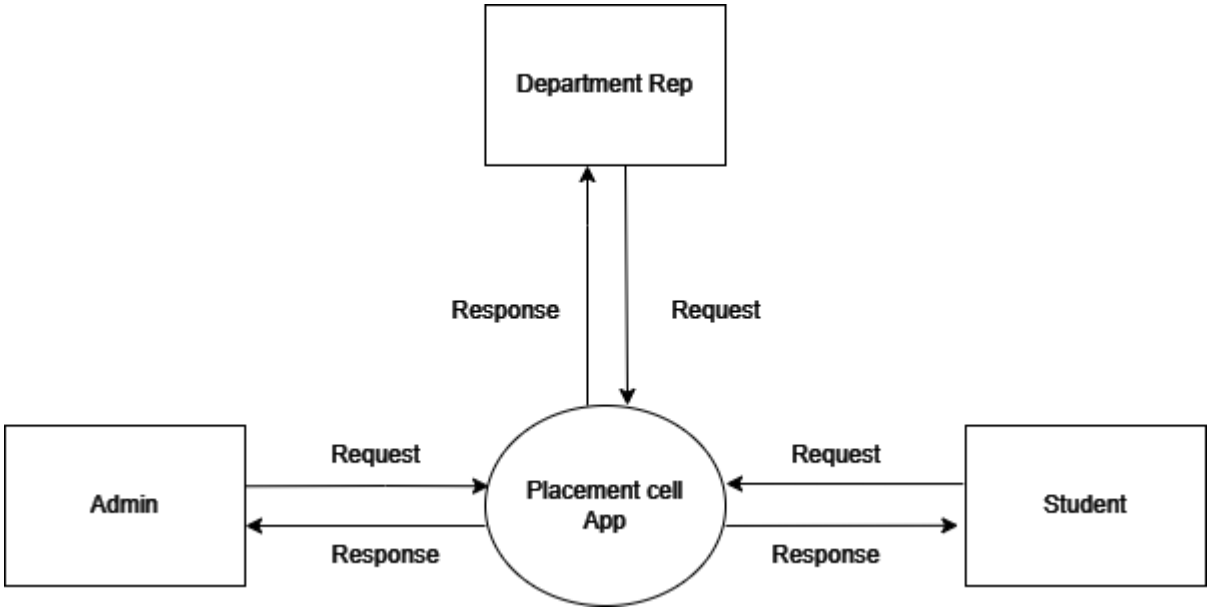
Data flow - Identifies data flow Circle

Bubble - Represents a process that transforms incoming data to outgoing data.

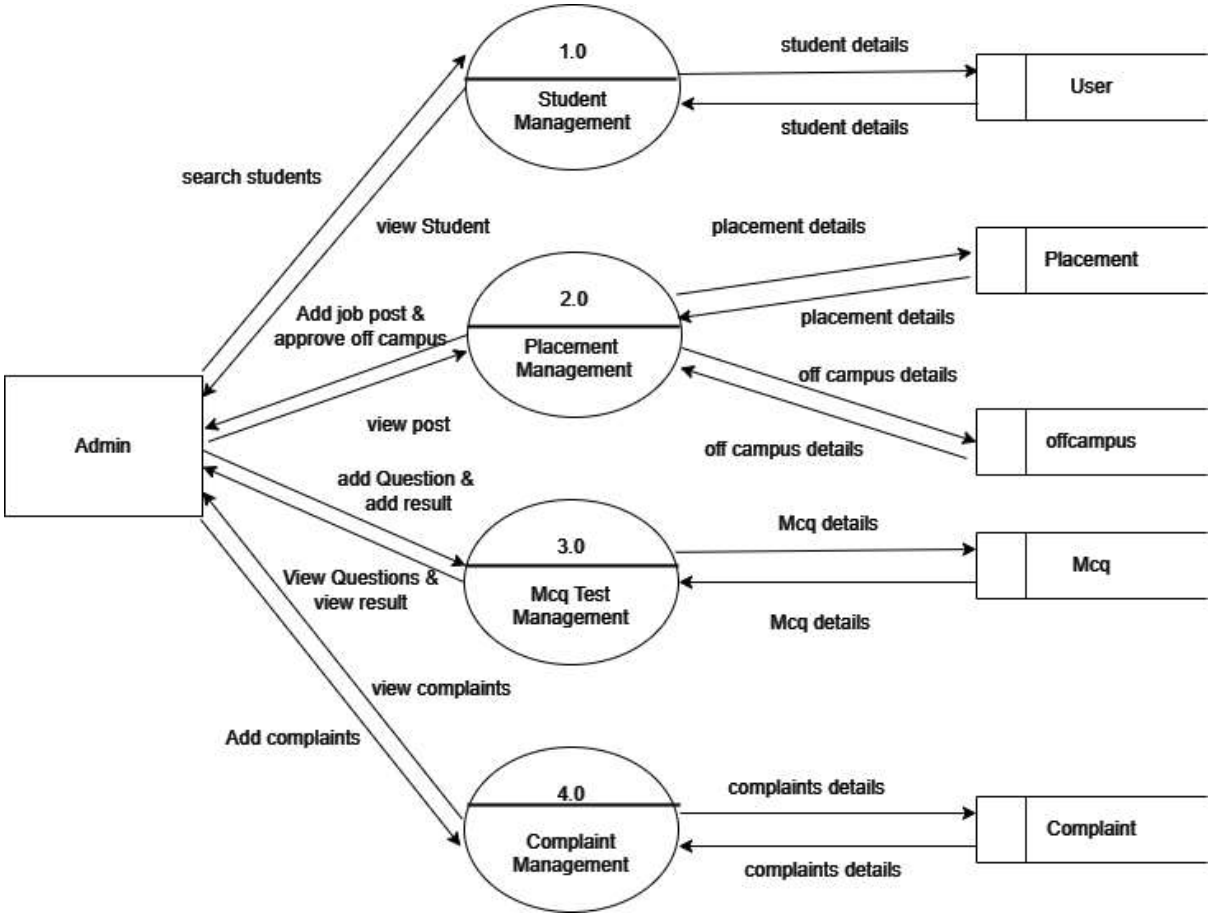


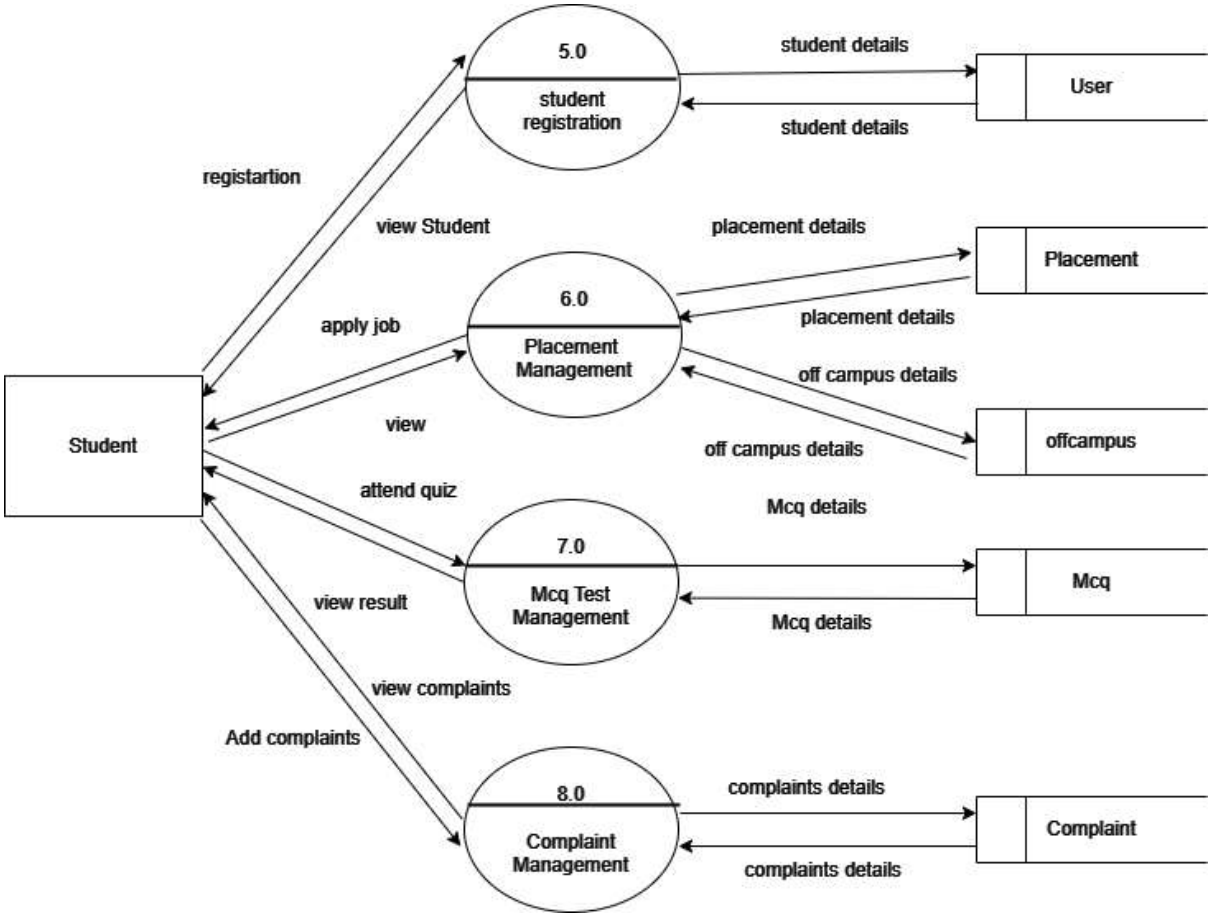
Open rectangle- Data store

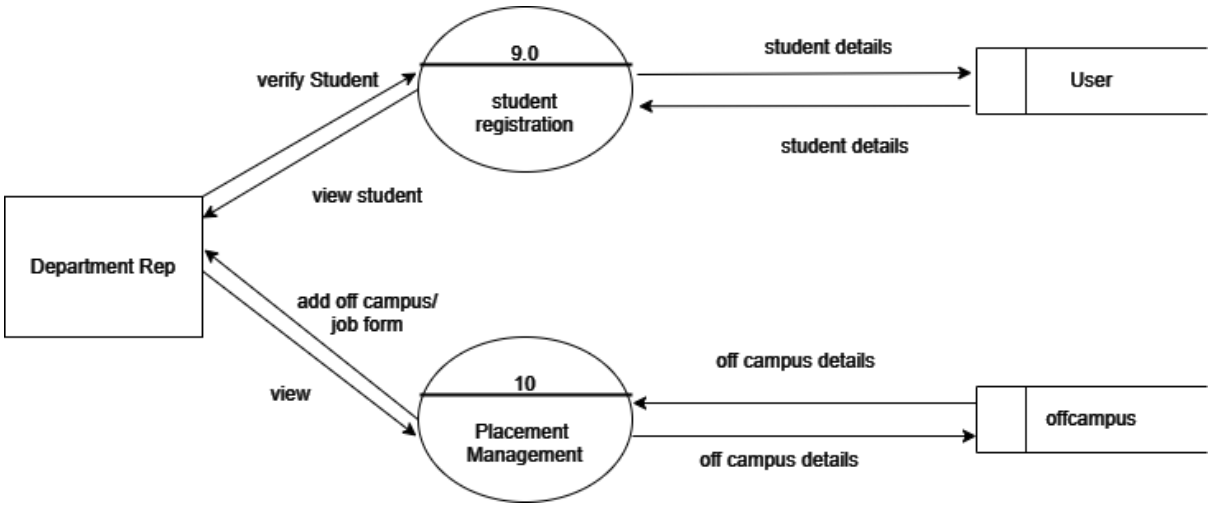
Level - 0


Department Rep
Response
Request
Admin
Request
Response
Placement cell App
Request
Response
Student

Level -1


1.0
Student Management
2.0
Placement Management
3.0
Mcq Test Management
4.0
Complaint Management
Admin
User
Placement
offcampus
Mcq
Complaint
student details
student details
search students
view Student
placement details
placement details
Add job post & approve off campus
off campus details
off campus details
view post
add Question & add result
Mcq details
Mcq details
View Questions & view result
view complaints
Add complaints
complaints details
complaints details


5.0
student registration
student details
student details
User
registartion
view Student
placement details
Placement
6.0
Placement Management
placement details
apply job
off campus details
view
off campus details
offcampus
Student
attend quiz
Mcq details
7.0
Mcq Test Management
Mcq
view result
Mcq details
view complaints
Add complaints
8.0
Complaint Management
complaints details
Complaint
complaints details


9.0
student registration
verify Student
view student
student details
student details
User
Department Rep
add off campus/ job form
view
10
Placement Management
off campus details
off campus details
offcampus

3. SYSTEM DESIGN

3.1 MODULE DESCRIPTION

- **Admin:** student management, department management, add job post, complaint management.
- **Student:** apply jobs and management, attend quiz, view result, add complaint academic profile registration.
- **Department rep:** Student management, add of campus placement.

3.2 INPUT DESIGN

The quality of the system input determines the quality of the system output. Input specification describes the manner in which data enter the system for processing. Input design features can ensure the reliability of the system and produce result from accurate data, or they can result in the production or erroneous information. The input design also determines whether the user can interact efficiently with the system. In our system almost, all inputs are being taken from the databases. To provide adequate inputs we have to select necessary values from the databases and arrange it to the appropriate controls.

3.3 OUTPUT DESIGN

One of the important features of an information system for users is the output produces. Output is the information delivered to users through the information system. Without quality of the output, the entire system appears to be unnecessary that users will avoid using it. Users generally merit the system solely by its output. In order to create the most useful output possible. One works closely with the user though an interactive process. until the result is considered to be satisfactory.

3.4 DATABASE DESIGN

Database design for an add-on project involves several key steps to ensure efficient data management and system functionality. Initially, it's crucial to identify the project's data requirements, including the types of data to be stored and their relationships. This leads to the conceptual design phase, where an abstract model of the database structure is created using tools like entity-relationship diagrams (ERDs). The logical design stage translates this conceptual model into a practical schema, defining tables, columns, keys, and constraints while normalizing the database to minimize redundancy. The physical design considers hardware and software aspects, optimizing storage structures, indexing, and performance strategies. Data integrity measures, including constraints and validation mechanisms, are implemented alongside security measures like access controls and encryption. Scalability and performance considerations, such as sharding and query optimization, ensure the database can handle increasing loads efficiently.

Table name: User

Description: This tables stores the User details

Field	Type	Description
_id	String	Automatic Generated Id
name	String	Name of the user
email	String	Email of the user
phone	Number	Phone number of the user
password	Number	Password of the user
usertype	String	User type of user
login_status	number	Login status of user

Table name: Department_rep

Description: Student rep details

Field	Type	Description
_id	Object id	Automatic Generated Id
userid	Object id	Userid of user
department	String	Department

Table: student_mark_list

Description: Stores the appointment details

Field	Type	Description
_id	Object id	Automatic Generated Id
userid	Object id	Id of the user
high_school_type	String	High school type
high_school_name	String	High school name
high_school_mark	String	High school mark
high_school_mark_list	String	High school mark list
higher_secondary_type	String	Higher secondary type
higher_secondary_name	String	Higher secondary name
higher_secondary_mark	String	Higher secondary mark
higher_secondary_mark_list	String	Higher secondary mark list
degree_course_name	String	Degree course name
degree_college_name	String	Degree college name
degree_course_mark	String	Degree course mark
degree_course_mark_list	String	Degree course mark list
current_reg_no	String	Current register number
department	String	Department name
status	String	Status

Table: student_personal_details

Description: Stores the personal details

Field	Type	Description
_id	Object id	Automatic Generated Id
userid	userid	Id of the user
housename	String	Student house name
city	String	Student city
district	String	Student district
pin code	number	Pin code of the city

Table: result

Description: Stores the details of result

Field	Type	Description
_id	Object id	Automatic Generated Id
exam_name	string	category
exam_description	string	Exam description
result_doc	string	result
uploaddate	date	Upload date

Table: placements

Description: Stores the placement details

Field	Type	Description
_id	Object id	Automatic Generated Id
companyname	string	Company name
designation	string	Job designation
package	number	Job package
jobdescription	string	Job description
applicable_departments	array	Applicable departments
cuttoff	string	Mark cut off
link	string	Link of placement
enddate	string	End date of placement

Table: placement_join

Description: Stores the joinees details

Field	Type	Description
_id	Object id	Automatic Generated Id
studentid	Object id	Id of the student
placementid	String	Id of the placement
joindate	String	Join date

Table: offcampus

Description: Stores the details of off campuses

Field	Type	Description
_id	Object id	Automatic Generated Id
userid	Object id	Id of the user
companyname	String	Company name
designation	String	Designation
package	String	Package of the job
jobdescription	String	Description of the job
applicable_departments	Array	Applicable departments
link	String	Link to apply
admin_status	String	Admin status

Table: Mcq

Description: Stores the details of mcq

Field	Type	Description
_id	Object id	Automatic Generated Id
questionText	string	Question for mcq
options	array	Options of the question
correctOptionIndex	string	Correct option

Table: Complaint

Description: Stores the details of complaint

Field	Type	Description
_id	Object id	Automatic Generated Id
userid	Object id	User details
subject	string	subject
complaint	string	About the complaint
reply	string	Reply for complaint
timestamp	number	Timestamp
status	String	Status of complaint

4. SYSTEM TESTING & IMPLEMENTATION

4.1 SYSTEM TESTING

Testing plays a vital role in the development of the Presence Pro, serving as a quality assurance process to validate the app's functionality and reliability. The testing team will conduct various levels of testing, including unit testing to verify individual components, integration testing to assess interactions between modules, and user acceptance testing to evaluate the app from the end users' perspective. Manual and automated testing techniques will be employed to catch defects early, ensure compliance with requirements, and optimize the app's performance.

System testing is defined as the process by which one detects the defects in the software. Any software development organization or team has to perform several processes. Software testing is one among them. It is the final opportunity of any programmer to detect and rectify any defects that may have appeared during the software development stage. Testing is a process of testing a program with the explicit intention of finding errors that makes the program fail. In short system testing and quality assurance is a review in software products and related documentation for completion, correctness, reliability and maintainability.

System testing is the first stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is vital to the success of the system. System testing makes a logical assumption that if all the parts of the system are correct and the goal will be successfully achieved. A series of testing are performed for the proposed system before the proposed system is ready for user acceptance testing.

The testing steps are,

• Unit Testing

• Integration Testing

• Validation testing

• Output Testing

• Acceptance Testing

System Testing provides the file assurance that software once validated mast combined with all other system elements. System testing verifies whether all elements nave been combined

properly and that overall system function and performance is achieved. FA the integration of modules, the validation test was carried out over the system.

lt was that all the modules work well together and meet the overall system function and performance. Unit Testing Unit testing is carried out screen-wise, each screen being identified as an object. Attention is diverted to individual modules, independently to one another to locate errors. This has enabled the detection of errors in coding and logic. Various test cases are prepared. For each module these test cases are implemented and it is checked whether the module is executed as per the requirements and outputs the desired result. In this test each service input and output parameters are checked.

Unit testing

- Module interface was tested to ensure that information properly flows into and out of the program under test.
- Boundary condition was tested to ensure that module operates properly at boundaries established to limit or restrict processing.
- All independent paths through the control structures were executed to ensure that all statements in the modules have been executed at least once.
- Error handling paths were also tested.

Integration Testing

Integration testing is a systematic technique for constructing the program structure while at the same time conducting tests to uncover errors associated with interfacing.

Unit tested module were taken and a single program structure was built that has been dictated by the design. Incremental integration has been adopted here. The modules are tested separately lor accuracy and modules are integrated too.th tn. using bottom-up integration i.e., by integrating from moving from bottom to the toon the system is checked and errors found during integration are rectified.

In this testing individual modules were combined and he m0duie wise Shifting was verified to be alright.The entire software was developed and tested in small segments, where errors were easy to locate and rectify. Program builds (group of modules) were constructed corresponding to the successful testing of user interaction, data manipulation analysis, and display processing and database management.

Validation Testing

Validation testing is done to ensure complete assembly of the error-free software. Validation can be termed successful only if it functions in manner. Reasonably expected by the student under validation is alpha and beta testing. The student-side validation is done in this testing phase. It is checked whether the data passed to each student is valid or not. Entering incorrect values does the validation testing and it is checked whether the errors are being considered. Incorrect values are to be discarded. The errors are rectified. In “Placement cell App" verifications are done correctly. So, there is no chance for users to enter incorrect values. It will give error messages by using different validations. The validation testing is done very clearly and found it is error free.

Output Testing

After performing the validation testing the next step is output testing of the proposed system, since no system could be useful if it does not produce the required output in a specific format. The output format on the screen was found to be correct as the format was designed in the system design phase according to the user needs. For the hard copy also, the output comes out as specified requirement by the user. Hence output testing does not result in any Correction in the system. output This project is developed based on the user choice. It is user friendly. The output format is very clear to user. Output testing is done on Smart builders correctly.

Acceptance testing

Acceptance involves running a suite of tests on the completed system. Each individual test, known as a Case, exercise particular operating condition of the operating condition of the user's environment or feature of the system, and will result in a pass fail, or Boolean outcome.

4.2 SYSTEM IMPLEMENTATION

The implementation is the final state and it is an important phase. It involves the invalid programming system testing. user training and the operational running of developed proposed system that constitutes the application subsystems. A major task of preparing for implementation is education of users. which should really have been taken place much carrier in the project when they were belong involved in the investigation and design work. During the implementation phase system actually take physical shape. In order to develop a system implemented planning is very essential. The implementation phase of the software development is concerned with translating design specification into source code. The user tests the developed system and changes are made according to their needs. Our system has been successfully implemented. Before implementation several tests have been conducted to ensure that no errors are encountered during the operation. The implementation phase ends with an evaluation of the system after placing into the operation for a period of time. The process of putting the developed system in actual use is called system implementation. This includes all those activities that take place to convert from old system to new system. The system can be implemented only after testing is done and is found to be working to specifications. The implementation stage is a systems project in its own right.

The implementation stage involves following tasks:

- Careful planning.
- Investigation of system and constraints.
- Design of method to achieve change over
- Evaluation of the changeover method.

In the case of this project all the screens are designed first. For making it to be executable, codes are written on each screen and performs the implementation by creating the database and connecting to the server. After that the system, is Checked, whether it performs all the transactions Correctly. Then databases are cleared and made it to be usable to the technicians.

5. SECURITY TECHNOLOGIES AND POLICTES

The protection of computer-based resources that includes hardware, software, data procedures and people against unauthorized use or natural. Disaster is known as System Security.

System Security can be divided into four related issues:

- Security
- Integrity
- Privacy
- Confidentiality

SYSTEM SECURITY refers to the technical innovations and procedures applied to the hardware and operation systems to protect against deliberate or accidental damage from a defined threat.

DATA SECURITY is the protection of data from loss, disclosure, modification and destruction.

SYSTEM INTEGRITY refers to the power functioning of hardware and programs, appropriate physical security and safety against external threats such as caves dropping and wiretapping

PRIVACY defines the rights of the user or organizations to determine what information they are willing to share with or accept from others and how the organization can be protected against unwelcome, unfair or excessive dissemination of information about it.

CONFIDENTIALITY is a special status given to sensitive information in a database to minimize the possible invasion of privacy. It is an attribute of information that characterizes it needs for protection.

SECURITY IN SOFTWARE System security refers to various validations on data in form of checks and controls to avoid the system from failing. It is always important to ensure that only valid data is entered and only valid operations are performed on the systems.

The system employees two types check and controls:

CLIENT-SIDE VALIDATION Various client-side validations are used to ensure on the client side that only valid data is entered. Client-side validation saves server time and load to handle invalid data.

Some checks imposed are:

- Forms cannot be submitted without filling up the mandatory data so that manual mistakes of submitting empty fields that are mandatory can be sorted out at the client side to save the server time and load.
- Tab-indexes are set according to the need and taking into account the ease of user while working with the system.

SERVER-SIDE VALIDATION Some checks cannot be applied at client side. Server-side checks are necessary to save the system from failing and intimating the user that some invalid operation has been performed or the performed operation is restricted. Some of the server-side checks imposed is:

- Server-side constraint has been imposed to check for the validity of primary key and foreign key. A primary key value cannot be duplicated. Any attempt to duplicate the primary value results into a message intimating the user about those values through the forms using foreign key can be updated only of the existing foreign key values.
- User is intimating through appropriate messages about the successful operations or exceptions occurring at server side.
- Various Access Control Mechanisms have been built so that one user may not agitate upon another. Access permissions to various types of users are controlled according to the organizational structure. Only permitted users can log on to the system and can have access according to their category. User name, passwords and permissions are controlled over the server side.
- Using server-side validation, constraints on several restricted operations are imposed

6. MAINTENANCE

Software maintenance is the modification of a software product aner delivery to correct faults, to improve performance or other attributes. Maintenance is the ease with which a program can be corrected if any error is encountered, adapted if its environment changes or enhanced if the customer desires a change in requirement. Maintenance follows conversation to extend that changes are necessary to maintain satisfactory operations relative to changes in the user's environment.

Maintenance often includes minor enhancements or corrections to problems that surface in the system's operation. Maintenance is also done based on fixing the problems reported, changing the interface with other software or hardware enhancing the software.

CATEGORIES OF MAINTENANCE

Corrective Maintenance

Corrective maintenance is the most commonly used maintenance approach, but it is easy to see its limitations. When equipment fails, it often leads to downtime in production, and sometimes damages other parts. In most cases, this is expensive. Also, if the equipment needs to be replaced, the cost of replacing it alone can be substantial. Reliability of systems maintained by this type of maintenance is unknown and cannot be measured. Corrective maintenance is possible since the consequences of failure or wearing out are not significant and the cost of this maintenance is not great.

Adaptive Maintenance

Modification of a software product performed after delivery to keep a are product usable m a changed or changing environment. Adaptive maintenance includes any work initiated as a consequence of moving the software to a different hardware or software platform. It is a change driven by the need to accommodate modifications in the environment of software system. The environment in this context refers to the totality of all conditions and influences which act from outside upon the system. A change to the whole or part of this environment will Warrant a corresponding modification of the software.

Perfective Maintenance

Modification of a software product alter delivery to improve performance or maintainability. This term is used to describe changes undertaken to expand the existing requirements of the system. A successful piece or software lends to be subjected to the Succession of changes resulting in an increase in us requirements. This is based a premise that as the software becomes useful, the user experiment with new cases beyond the of Scope for which it was initially developed. Vxpansi01 n requirements can take the form enhancement of existing system functionality and improvement in computational efficiency.

Preventive Maintenance

Preventive maintenance is a schedule of planned maintenance actions aimed at the prevention of breakdowns and failures. The primary goal of preventive maintenance is to prevent the failure of equipment before it actually occurs. It is designed to preserve and enhance equipment reliability by replacing worn components before they actually fail. Preventive maintenance activities include equipment checks, partial or complete overhauls at specified periods.

Long-term benefits of preventive maintenance include:

• Improved system reliability.

• Decreased cost of replacement.

• Decreased system downtime.

7. SCOPE FOR FUTURE ENHANCEMENT

The Placement Cell Application has been developed with the goal of streamlining the placement process and improving the overall efficiency and effectiveness for students, volunteers, and placement coordinators. As the application matures and gains user feedback, there are several opportunities for future enhancement and scope of further development. This chapter explores the potential areas of improvement to make the application even more robust and user-friendly.

Merits of the System

1. Streamlined Placement Process: The application offers a centralized platform that simplifies the placement process for students, volunteers, and coordinators. It facilitates easy communication, data management, and real-time updates.

2. User-Friendly Interface: The intuitive and user-friendly interface ensures that users can quickly navigate through the application and access the required functionalities with ease.

3. Efficient Communication: The application provides a smooth channel of communication between students, volunteers, and coordinators, leading to effective coordination and timely updates.

4. Scalability: The use of Flutter for frontend development and Node.js for backend development ensures the application's scalability to handle increasing user demands and data loads.

5. Enhanced Data Management: The application efficiently manages student profiles, placement details, and communication records, making the entire process more organized.

8. CONCLUSION

The Placement Cell App is a comprehensive mobile application meticulously designed to optimize and expedite the often complex and multifaceted placement process for students, volunteers, and placement coordinators. Its core purpose is to address the myriad challenges encountered in efficiently orchestrating placement activities. This multifunctional app manifests as an intuitive and accessible platform, adept at fostering seamless communication, centralizing data management, and propelling the entire placement landscape towards a more cohesive and organized structure. Forged with the amalgamation of cutting-edge technologies including Flutter, Node.js, and MongoDB, the app emerges as a potent tool equipped with not only user-friendliness but also a native-like experience. This amalgamation ensures a harmonious blend of a visually appealing and smooth interface with a robust backend that can adeptly manage a multitude of tasks such as job postings, student particulars, and real-time application tracking. The developmental trajectory of this app adheres to an incremental process model, allowing it to grow and enhance iteratively, leveraging user feedback for targeted improvements. This continuous evolution reflects the app's commitment to staying relevant and efficacious in the dynamic realm of placement activities.

9. BIBLIOGRAPHY

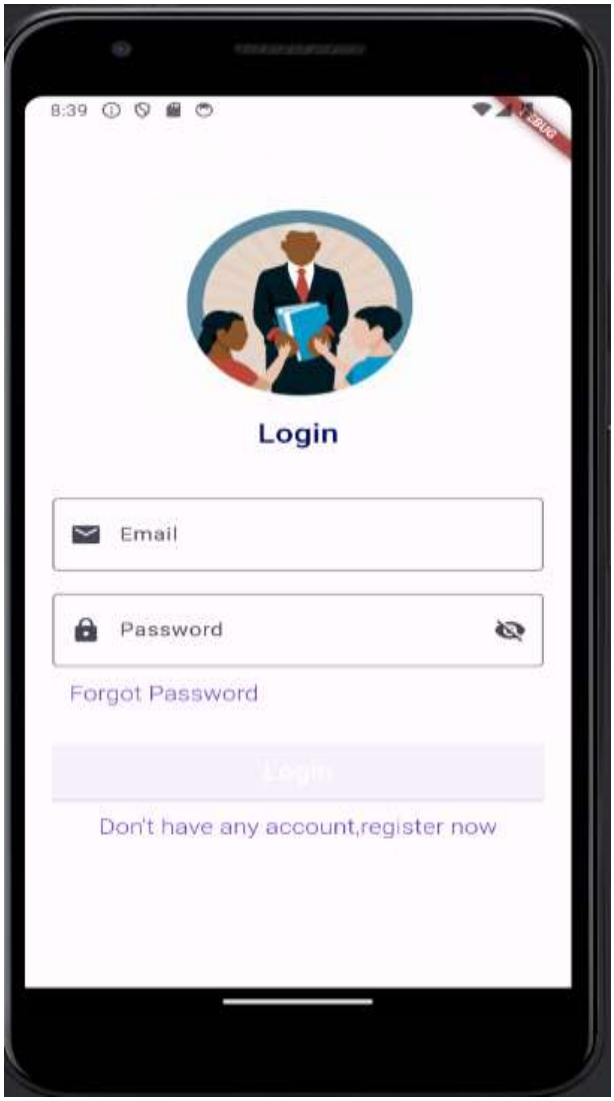
1. Flutter – Build apps for any screen
   https://flutter.dev/
2. The official repository for Dart and Flutter packages
   https://pub.dev/
3. https://www.tutorialspoint.com/nodejs/nodejs_express_framework.htm
4. https://www.w3schools.com/mongodb/
5. Figma – Designing Tool

   https://www.figma.com/

10. APPENDIX

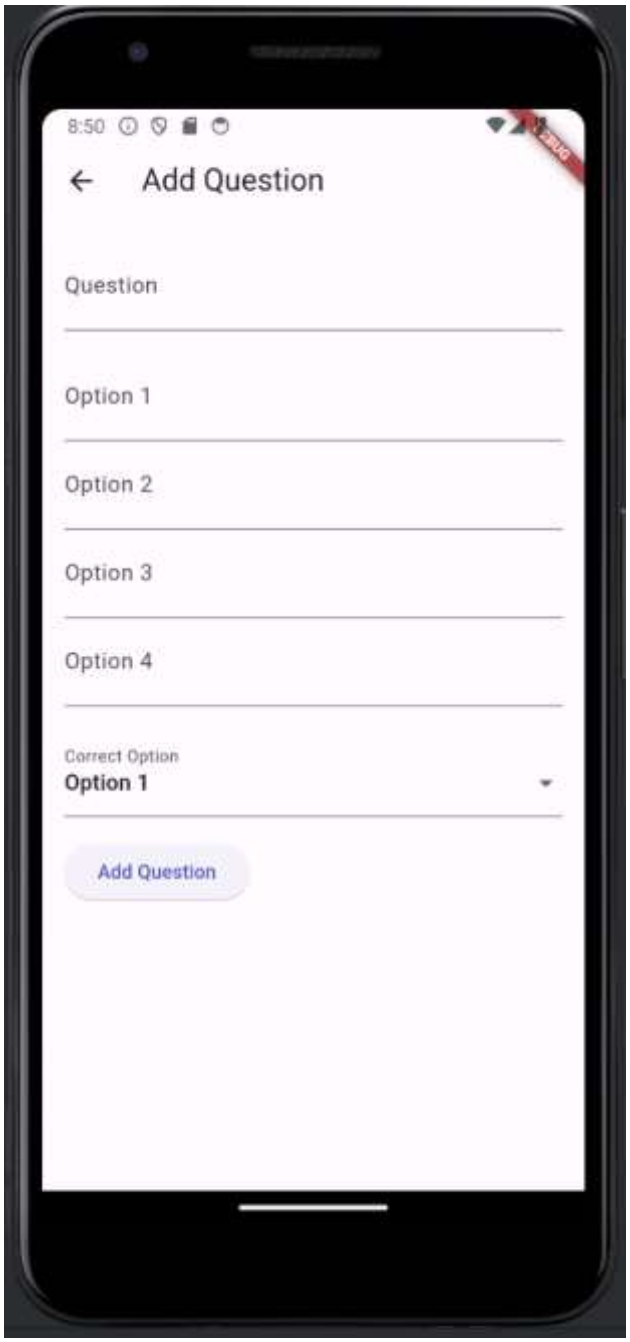
10.1 Screen Shots

Login

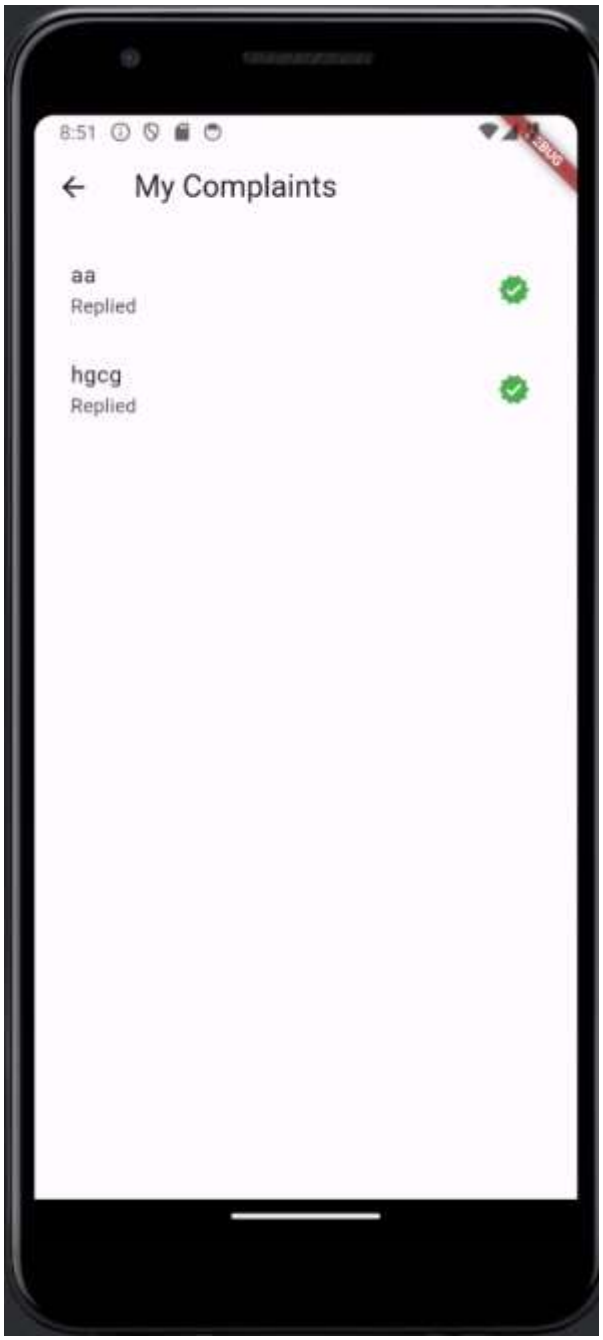


Admin dashboard

Add Question



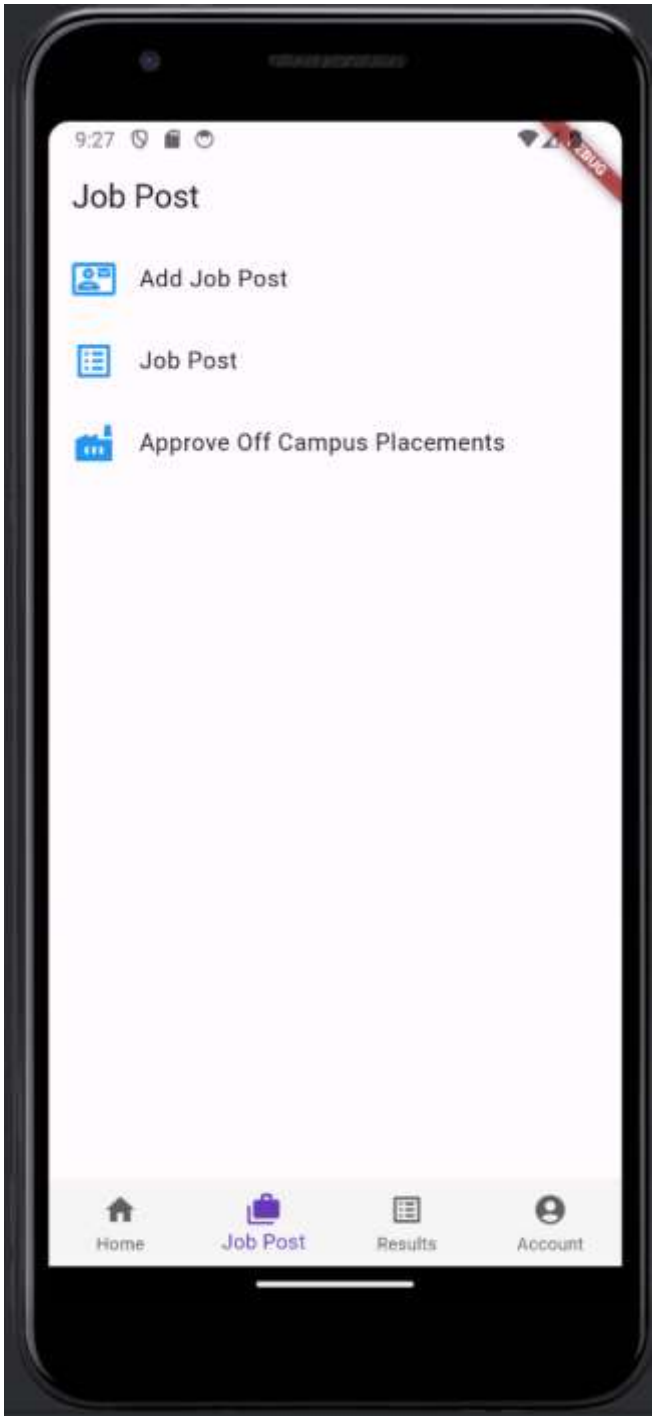
Add Complaint



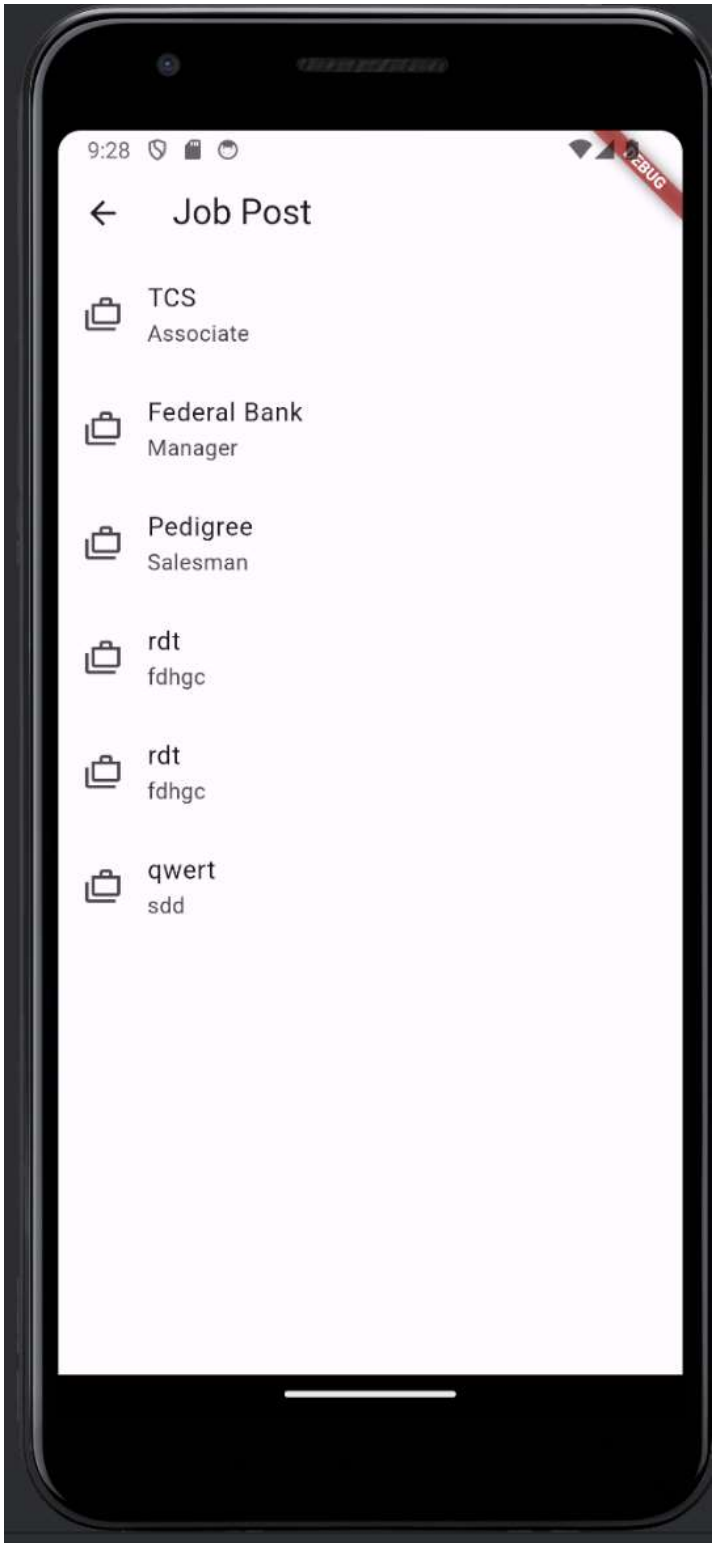
View result


9:07
Result
Add Result
View Result
Home
Job Post
Results
Account

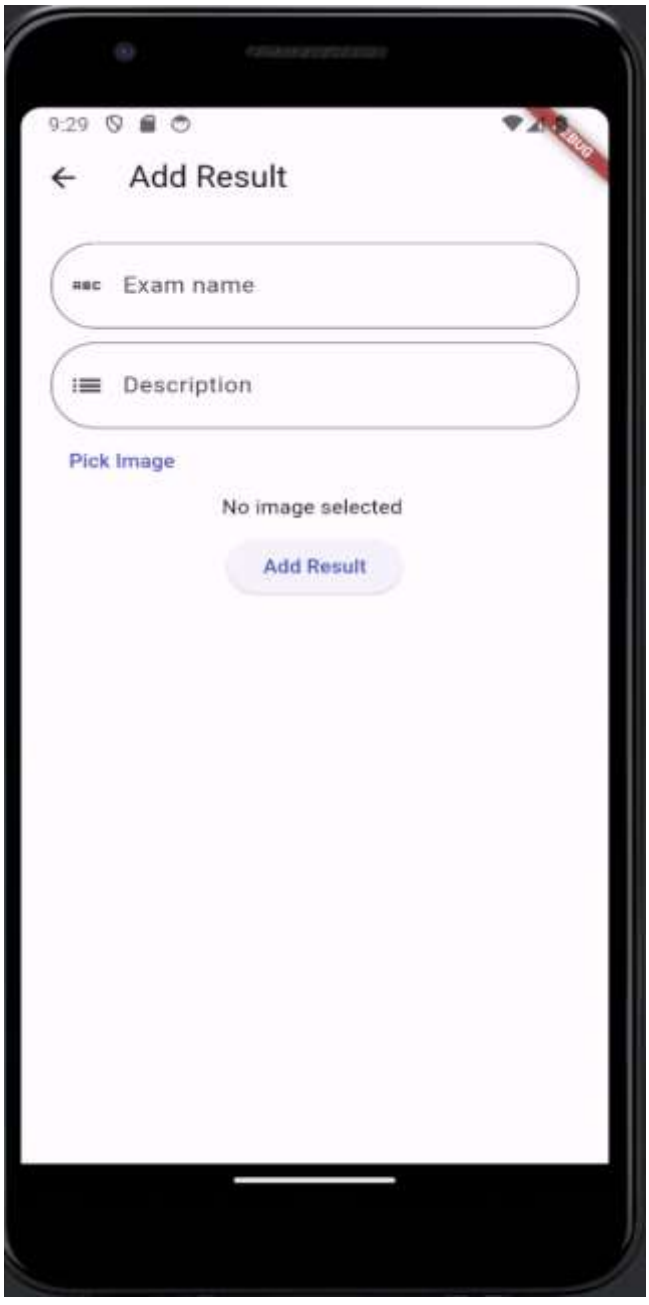
Job Post


9:27
Job Post
Add Job Post
Job Post
Approve Off Campus Placements
Home
Job Post
Results
Account

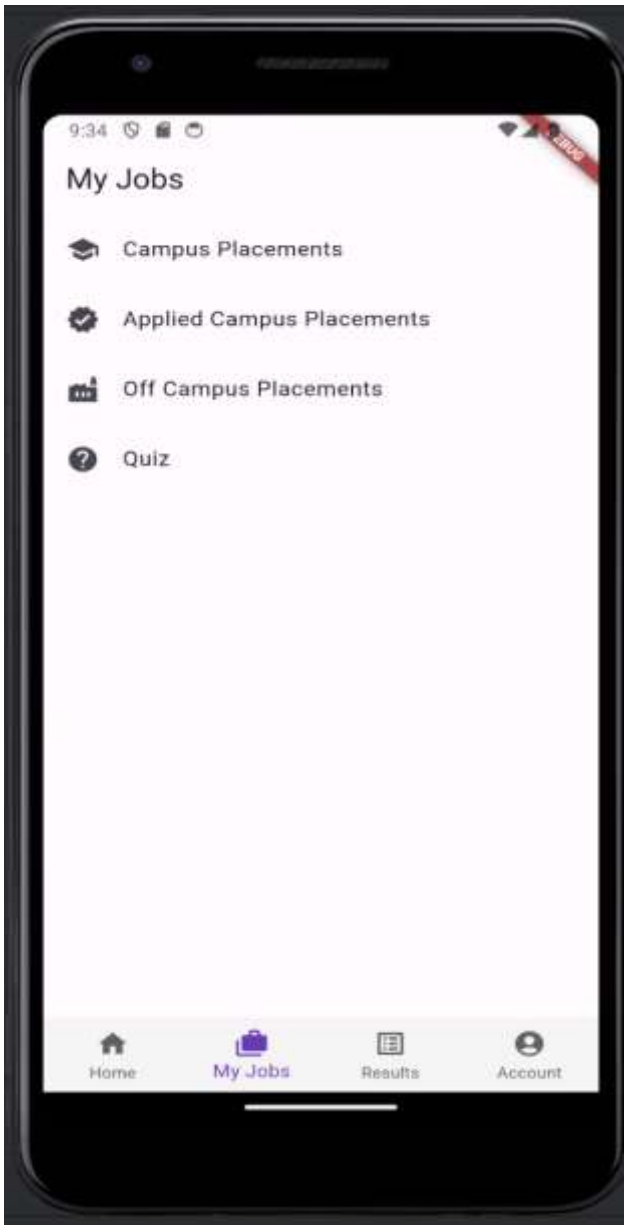
Job post


9:28
DEBUG
Job Post
TCS
Associate
Federal Bank
Manager
Pedigree
Salesman
rdt
fdhgc
rdt
fdhgc
qwert
sdd

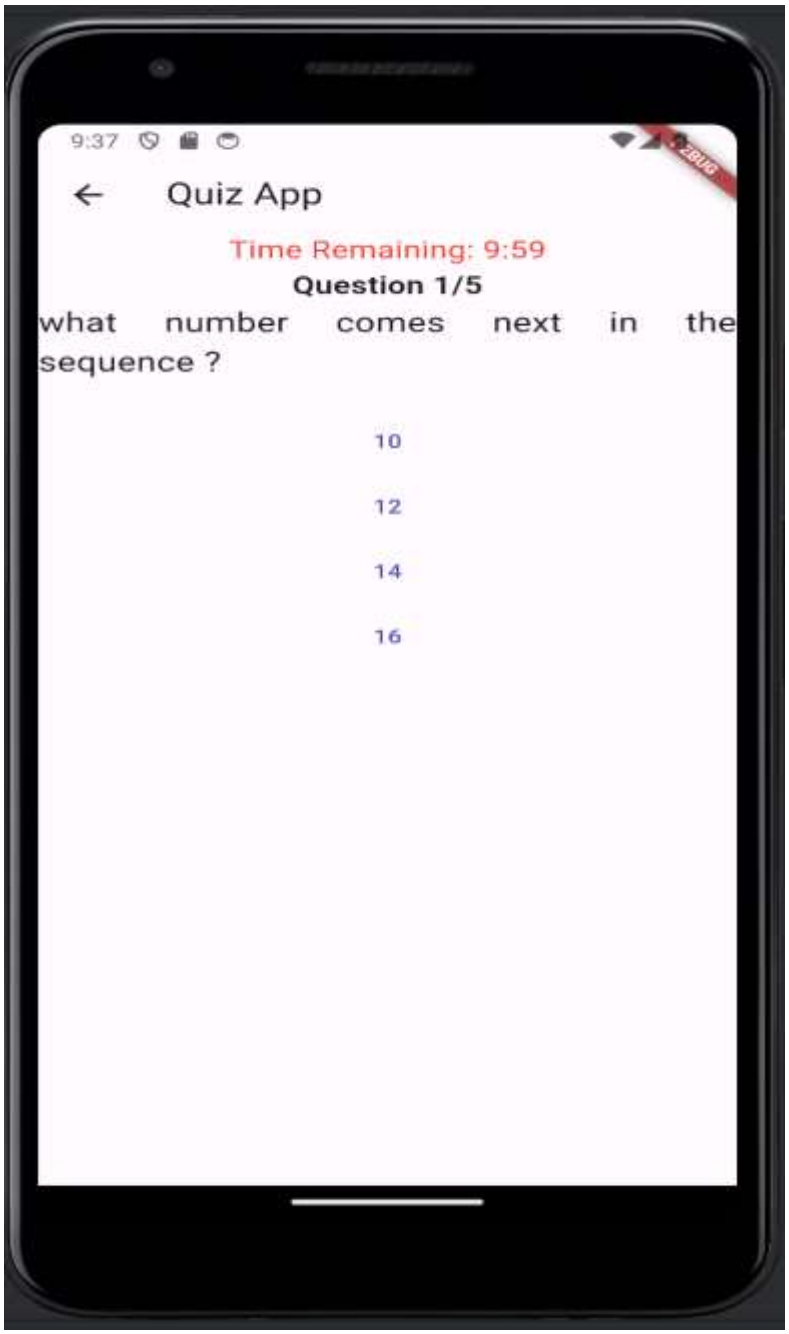
Add result


9:29
Add Result
Exam name
Description
Pick Image
No image selected
Add Result

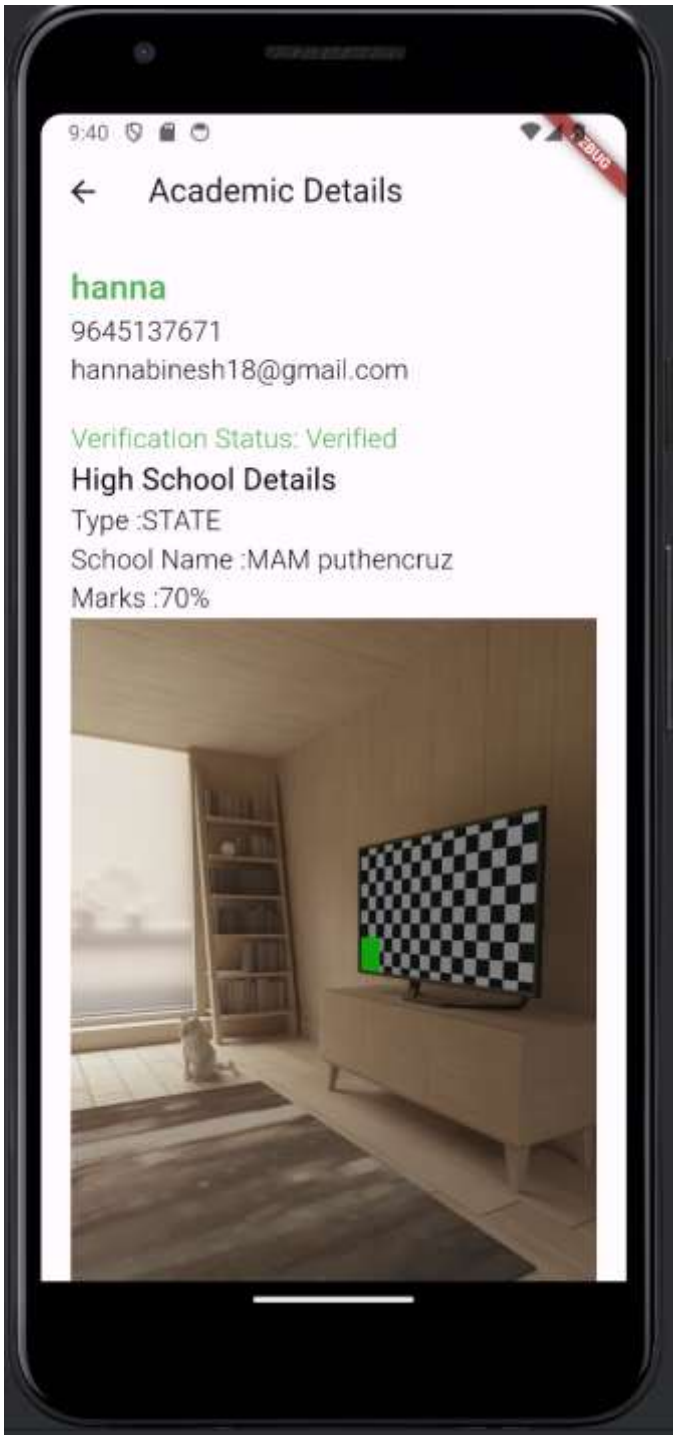
Department dashboard



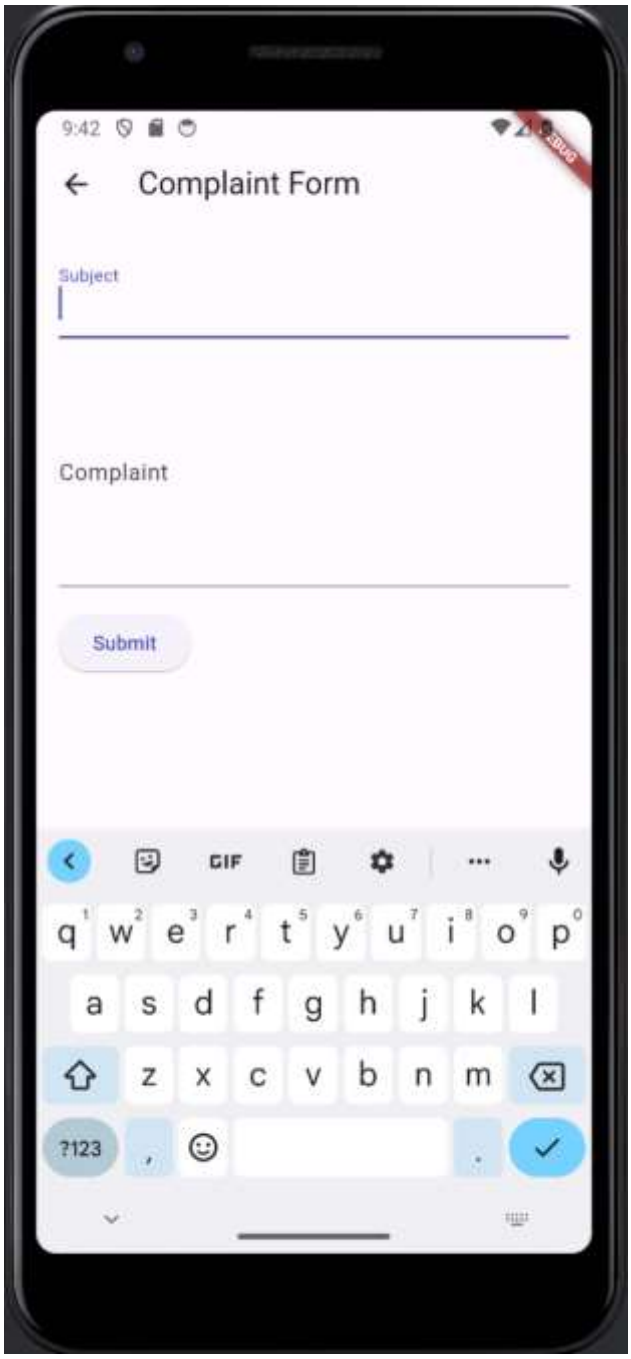
Quiz question



Profile


9:40
Academic Details
hanna
9645137671
hannabinesh18@gmail.com
Verification Status: Verified
High School Details
Type :STATE
School Name :MAM puthencruz
Marks :70%

Complaint



10.2 CODES

Profile

import 'dart:math';

```
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:placement_cell_app/services/authservice.dart';
import 'package:placement_cell_app/services/utilityservice.dart';

class ProfilePage extends StatefulWidget {
  const ProfilePage({super.key});
  @override
  State<ProfilePage> createState() => _ProfilePageState();
}
class _ProfilePageState extends State<ProfilePage> {
  final storage = const FlutterSecureStorage();
  bool isloading = false;
  dynamic _data;
  AuthService _authService = AuthService();
  TextEditingController _phone = TextEditingController();
  TextEditingController _email = TextEditingController();
  TextEditingController _password = TextEditingController();
  TextEditingController _newpassword = TextEditingController();
  UtilityService _utilityService = UtilityService();
  String? password;
```

```
bool istrue = false;


getProfile() async {
  if (mounted) {
    setState(() {
      isloading = true;
    });
  }
  Map<String, String> allValues = await storage.readAll();
  String? userid = allValues["userid"];
  try {
    final Response res = await _authService.getUser(userid!);
    if (mounted) {
      setState(() {
        _data = res.data;
        _email.text = res.data["email"];
        _phone.text = res.data["phone"].toString();
        password = res.data["password"];
        isloading = false;
      });
    }
  } on DioException catch (e) {
    setState(() {
      isloading = false;
    });
    ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
      content: Text("Error occurred,please try again"),
      duration: Duration(milliseconds: 300),
    ));
```

```
  }
 }

 updateEmail() async {
  Map<String, String> allValues = await storage.readAll();
  String? userid = allValues["userid"];
  try {
   final Response res =
       await _utilityService.updateEmail(_email.text, userid!);
   ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
    content: Text("Email Updated Successfully"),
    duration: Duration(milliseconds: 3000),
   ));
  } on DioException catch (e) {
   ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
    content: Text("Error occurred,please try again"),
    duration: Duration(milliseconds: 300),
   ));
  }
 }

 updatePhone() async {
  Map<String, String> allValues = await storage.readAll();
  String? userid = allValues["userid"];
  try {
   final Response res = await _utilityService.updatePhoneNumber(
       int.parse(_phone.text.toString()), userid!);
   ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
    content: Text("Phon Number Updated Successfully"),
    duration: Duration(milliseconds: 3000),
```

```
    ));
  } on DioException catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
      content: Text("Error occurred,please try again"),
      duration: Duration(milliseconds: 300),
    ));
  }
}

checkPassword() {
  if (password == _password.text) {
    setState(() {
      istrue = true;
    });
  } else {
    showDialog(
        context: context,
        builder: (BuildContext context) {
          return AlertDialog(
            title: Text("Password Not Coorect"),
            content: Text("Current password is not correct"),
            actions: [
              TextButton(
                child: Text("Ok"),
                onPressed: () {
                  Navigator.of(context).pop();
                },
              )
            ],
          );
```

```
        });
    }
  }

  changePassword() async {
    Map<String, String> allValues = await storage.readAll();
    String? userid = allValues["userid"];
    try {
      final Response res =
          await _utilityService.updatePassword(_newpassword.text, userid!);
      ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
        content: Text("Password Updated Successfully"),
        duration: Duration(milliseconds: 3000),
      ));
    } on DioException catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
        content: Text("Error occurred,please try again"),
        duration: Duration(milliseconds: 300),
      ));
    }
  }

  @override
  void initState() {
    // TODO: implement initState
    getProfile();
    super.initState();
  }

  @override
```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("My Profile")),
    body: isloading
      ? const Center(
        child: CircularProgressIndicator(),
       )
      : ListView(
        padding: EdgeInsets.all(20),
        children: [
         Text(
          _data["name"],
          style: TextStyle(
            fontSize: 18,
            color: Colors.blue,
            fontWeight: FontWeight.w600),
         ),
         Text(
          _data["email"],
          style: TextStyle(
            fontSize: 18,
            color: Colors.black,
            fontWeight: FontWeight.w600),
         ),
         SizedBox(
          height: 10,
         ),
         TextFormField(
          controller: _email,
          decoration: InputDecoration(label: Text("Email")),
```

```
  keyboardType: TextInputType.emailAddress,
  validator: (p0) {
   if (p0!.isEmpty) {
    return "Please enter email";
   }
   return null;
  },
),
Container(
   child: ElevatedButton(
      onPressed: () {
       updateEmail();
      },
      child: Text("Update Email"))),
SizedBox(
 height: 10,
),
Text(
 _data["phone"].toString(),
 style: TextStyle(
     fontSize: 18,
     color: Colors.black,
     fontWeight: FontWeight.w600),
),
SizedBox(
 height: 10,
),
TextFormField(
 controller: _phone,
 decoration: InputDecoration(label: Text("Phone")),
```

```
  keyboardType: TextInputType.phone,
  validator: (p0) {
    if (p0!.isEmpty) {
      return "Please enter phone number";
    }
    return null;
  },
),
Container(
    child: ElevatedButton(
        onPressed: () {
          updatePhone();
        },
        child: Text("Update Phone"))),
SizedBox(
  height: 10,
),
Text(
  "Change Password",
  style: TextStyle(fontSize: 16),
),
TextFormField(
  controller: _password,
  decoration:
      InputDecoration(label: Text("Current Password")),
  keyboardType: TextInputType.visiblePassword,
  validator: (p0) {
    if (p0!.isEmpty) {
      return "Please enter password";
    }
```

```
      return null;
    },
  ),
  ElevatedButton(
      onPressed: () {
        checkPassword();
      },
      child: Text("Submit")),
  SizedBox(
    height: 10,
  ),
  if (istrue) ...[
    Text(
      "New Password",
      style: TextStyle(fontSize: 16),
    ),
    TextFormField(
      controller: _newpassword,
      decoration: InputDecoration(label: Text("New Password")),
      keyboardType: TextInputType.visiblePassword,
      validator: (p0) {
        if (p0!.isEmpty) {
          return "Please enter password";
        }
        return null;
      },
    ),
    ElevatedButton(
        onPressed: () {
          changePassword();
```

```
                },
                child: Text("Password Changes"))
            ]
          ],
        ));
  }
}
```

Admin account

```
import 'package:flutter/material.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class AdminAccount extends StatefulWidget {
  const AdminAccount({super.key});

  @override
  State<AdminAccount> createState() => _AdminAccountState();
}

class _AdminAccountState extends State<AdminAccount> {
  final storage = FlutterSecureStorage();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: ListView(children: [
        ListTile(
          leading: const Icon(
            Icons.logout,
          ),
```

```
        title: const Text("Logout"),
        onTap: () async {
          await storage.delete(key: "token");
          Navigator.of(context).pushNamedAndRemoveUntil(
              '/login', (Route<dynamic> route) => false);
        },
        selectedColor: Colors.green[800],
      ),
    ]),
  );
 }
}
```

Admin dashboard

```
import 'package:flutter/material.dart';

class AdminDashboard extends StatefulWidget {
 const AdminDashboard({super.key});

 @override
 State<AdminDashboard> createState() => _AdminDashboardState();
}

class _AdminDashboardState extends State<AdminDashboard> {
 @override
 Widget build(BuildContext context) {
  return Scaffold(
    body: ListView(
      children: [
       const SizedBox(
```

```
  height: 20,
 ),
 Padding(
  padding: const EdgeInsets.all(8.0),
  child: Row(
   mainAxisAlignment: MainAxisAlignment.spaceBetween,
   children: [
    GestureDetector(
     onTap: () {
      Navigator.pushNamed(context, "/view_all_students");
     },
     child: Container(
      width: 100,
      height: 100,
      decoration: BoxDecoration(
        color: Colors.deepPurple,
        borderRadius: BorderRadius.circular(10)),
      child: const Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
         Icon(
          Icons.group,
          size: 40,
          color: Colors.amber,
         ),
         Text(
          "Students",
          style: TextStyle(
            color: Colors.white,
            fontSize: 18,
```

```
                fontWeight: FontWeight.bold),
          )
        ]),
  ),
),
// const SizedBox(
//   width: 20,
// ),
// GestureDetector(
//   onTap: () {
//     Navigator.pushNamed(context, "/department_rep");
//   },
//   child: Container(
//     width: 100,
//     height: 100,
//     decoration: BoxDecoration(
//         color: Colors.deepPurple,
//         borderRadius: BorderRadius.circular(10)),
//     child: const Column(
//         mainAxisAlignment: MainAxisAlignment.center,
//         children: [
//           Icon(
//             Icons.person,
//             size: 40,
//             color: Colors.amber,
//           ),
//           Text(
//             "Department Rep",
//             textAlign: TextAlign.center,
//             style: TextStyle(
```

```
//                color: Colors.white,
//                fontSize: 18,
//                fontWeight: FontWeight.bold),
//           )
//         ]),
//   ),
// ),
SizedBox(
  width: 10,
),
GestureDetector(
  onTap: () {
    Navigator.pushNamed(context, "/add_mcq");
  },
  child: Container(
    width: 100,
    height: 100,
    decoration: BoxDecoration(
        color: Colors.deepPurple,
        borderRadius: BorderRadius.circular(10)),
    child: const Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(
            Icons.help,
            size: 40,
            color: Colors.amber,
          ),
          Text(
            "Add MCQ",
```

```
            textAlign: TextAlign.center,
            style: TextStyle(
              color: Colors.white,
              fontSize: 18,
              fontWeight: FontWeight.bold),
          )
        ]),
  ),
),
SizedBox(
  width: 10,
),
GestureDetector(
  onTap: () async {
    Navigator.pushNamed(context, "/view_all_complaint");
  },
  child: Container(
    width: 100,
    height: 100,
    decoration: BoxDecoration(
      color: Colors.deepPurple,
      borderRadius: BorderRadius.circular(10)),
    child: const Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(
          Icons.email,
          size: 40,
          color: Colors.amber,
        ),
```

```
                    Text(
                      "Complaints",
                      textAlign: TextAlign.center,
                      style: TextStyle(
                          color: Colors.white,
                          fontSize: 18,
                          fontWeight: FontWeight.bold),
                    )
                  ]),
              ),
            )
          ],
        ),
      ),
    ],
  ),
 );
 }
}
```

Admin Main

```
import 'package:flutter/material.dart';
import 'package:placement_cell_app/admin/add_job_post.dart';
import 'package:placement_cell_app/admin/add_result.dart';
import 'package:placement_cell_app/admin/admin_account.dart';
import 'package:placement_cell_app/admin/result_main.dart';
import 'package:placement_cell_app/admin/widgets/bottom_navigation.dart';

import 'admin_dashboard.dart';
```

```
final pages = [
  AdminDashboard(),
  AddJobPost(),
  ResultMain(),
  AdminAccount(),
];
final tites = ["Home", "Job Post", "Result", "Account"];

class AdminMain extends StatelessWidget {
  const AdminMain({super.key});

  @override
  Widget build(BuildContext context) {
    return ValueListenableBuilder(
      valueListenable: indexChanged,
      builder: (context, int index, child) {
        return Scaffold(
          appBar: AppBar(title: Text(tites[index])),
          bottomNavigationBar: const AdminBottomNav(),
          body: pages[index],
        );
      },
    );
  }
}
```

Repmain

```
import 'package:flutter/material.dart';
import 'package:placement_cell_app/representative/widgets/bottom_navigation.dart';
```

```
import 'package:placement_cell_app/students/jobresult.dart';
import 'package:placement_cell_app/students/myjobs.dart';
import 'package:placement_cell_app/students/student_dashboard.dart';
import 'package:placement_cell_app/students/studentaccount.dart';

final pages = [
  StudentDashboard(),
  MyJobs(),
  JobResult(),
  StudentAccount(),
];
final tites = ["Home", "My Jobs", "Result", "Account"];

class RepMain extends StatelessWidget {
  const RepMain({super.key});

  @override
  Widget build(BuildContext context) {
    return ValueListenableBuilder(
      valueListenable: indexChanged,
      builder: (context, int index, child) {
        return Scaffold(
          appBar: AppBar(title: Text(tites[index])),
          bottomNavigationBar: const RepBottomNav(),
          body: pages[index],
        );
      },
    );
  }
}
```

Viewcomplaint

```
import 'package:dio/dio.dart';
import 'package:flutter/material.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';
import 'package:placement_cell_app/services/utilityservice.dart';
import 'package:placement_cell_app/viewcomplaintsingle.dart';

class ViewAllComplaint extends StatefulWidget {
  const ViewAllComplaint({super.key});

  @override
  State<ViewAllComplaint> createState() => _ViewAllComplaintState();
}

class _ViewAllComplaintState extends State<ViewAllComplaint> {
  UtilityService _utilityService = UtilityService();
  List<dynamic> _data = [];
  final storage = const FlutterSecureStorage();
  bool isloading = false;

  getComplaints() async {
    if (mounted) {
      setState(() {
        isloading = true;
      });
    }
    Map<String, String> allValues = await storage.readAll();
```

```
  String? userid = allValues["userid"];
  try {
    final Response res = await _utilityService.getAllComplaint();


    print(res.data);
    if (mounted) {
      setState(() {
        _data = res.data;
        isloading = false;
      });
    }
  } on DioException catch (e) {
    setState(() {
      isloading = false;
    });
    ScaffoldMessenger.of(context).showSnackBar(const SnackBar(
      content: Text("Error occurred,please try again"),
      duration: Duration(milliseconds: 300),
    ));
  }
}


@override
void initState() {
  // TODO: implement initState
  getComplaints();
  super.initState();
}


@override
```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("My Complaints")),
    body: isloading
        ? const Center(
            child: CircularProgressIndicator(),
          )
        : _data.length == 0
            ? const Center(
                child: Text("No Complaints Added"),
              )
            : ListView.builder(
                padding: EdgeInsets.all(10),
                itemCount: _data.length,
                itemBuilder: (context, index) {
                  return ListTile(
                    title: Text(_data[index]["subject"]),
                    subtitle: Text(_data[index]["status"]),
                    trailing: _data[index]["status"] == "Pending"
                        ? Icon(
                            Icons.verified,
                            color: Colors.red,
                          )
                        : Icon(
                            Icons.verified,
                            color: Colors.green,
                          ),
                    onTap: () {
                      Navigator.push(
                        context,
```

```
                    MaterialPageRoute(
                        builder: (context) => ViewComplaintsSingle(
                            complaintid: _data[index]["_id"],
                        )),
                    );
                },
            );
        }));
  }
}
```